TransitionSet検索結果一覧 UI仕様書

概要

目的

セマンティック検索の結果を一覧表示し、各結果の詳細情報（スコア内訳、4軸テキスト）を展開表示するコンポーネント。

使用場面

- TransitionSetSearchModal内で検索結果を表示
- 各結果のスコア・プロジェクト名・ファイルパスを確認
- 詳細ボタンで4軸のテキストとスコア内訳を確認

アクセス権限

- **Experiment Admin（実験管理者）**: 検索結果の閲覧
- **その他のロール**: アクセス不可

画面構成

レイアウト構造

```
┌──────────────────────────────────────────────────┐
│ 50件 （125ms）                                   │
├──────────────────────────────────────────────────┤
│ #1 [transition_001] [ラマン] [Project001]  (85%) [詳細▼]│
│    /path/to/project/file.maiml                   │
│                                                  │
│    ┌─ 展開コンテンツ ──────────────────────┐  │
│    │ スコア内訳: [入力 90%] [手法 85%] [出力 80%] [構造 85%] │
│    │ ▼ Input テキスト                          │
│    │ ▼ Transition テキスト                     │
```

```
│    │ ▼ Output テキスト                                  │
│    └──────────────────────────────────────────────┘  │
├──────────────────────────────────────────────────────┤
│ #2 [transition_002] [XRD] [Project002]    (72%) [詳細▼]│
│    /path/to/project/file2.maiml                      │
└──────────────────────────────────────────────────────┘
```

UI要素

1. ヘッダー情報

- **表示内容**: 「{件数}件 （{検索時間}ms）」
- **フォントサイズ**: 12px
- **色**: グレー（secondary）

2. リストアイテム（各検索結果）

ヘッダー行:

要素	内容	スタイル
順位	#1、#2、...	太字、13px
TransitionIDタグ	transition_id	青色、`FileTextOutlined` アイコン
技術タグ	technique	紫色
プロジェクト名タグ	project_name	緑色、`FolderOpenOutlined` アイコン、クリック可能
スコア円形グラフ	score（0-100%）	サイズ36px、色はスコアに応じて変化
詳細ボタン	「詳細」	テキストボタン、`DownOutlined` アイコン（回転）

ファイルパス:

- **表示**: file_path
- **フォントサイズ**: 11px
- **色**: グレー（secondary）

展開コンテンツ:

- **背景色**: `#fafafa`
- **パディング**: 8px

- **ボーダー半径**: 4px
- **内容**:
  1. スコア内訳（4軸のスコア）
  2. 4軸テキスト（折りたたみパネル）

機能仕様

Props定義

```
interface SearchResultsProps {
  hits: SemanticSearchHit[];           // 検索結果
  loading: boolean;                    // ローディング状態
  error: string | null;                // エラーメッセージ
  searched: boolean;                   // 検索実行済みフラグ
  elapsedMs: number | null;            // 検索時間（ミリ秒）
  onOpenProject?: (projectName: string) => void; // プロジェクト名クリック時のコールバック
}
```

State管理

```
// 展開状態の管理（Set型）
const [expandedIds, setExpandedIds] = useState<Set<string>>(new Set());
```

主要機能

1. 展開/折りたたみの切り替え

```
const toggleExpand = (id: string) => {
  setExpandedIds((prev) => {
    const next = new Set(prev);
    if (next.has(id)) next.delete(id);
```

```
    else next.add(id);
    return next;
  });
};
```

動作:

- `expandedIds` にIDが存在する場合: 削除（折りたたむ）
- `expandedIds` にIDが存在しない場合: 追加（展開）

2. スコアを色に変換

```
function scoreToColor(score: number): string {
  if (score >= 0.8) return '#52c41a';  // 緑
  if (score >= 0.6) return '#1677ff';  // 青
  if (score >= 0.4) return '#faad14';  // オレンジ
  return '#ff4d4f';                    // 赤
}
```

スコア範囲と色:

スコア	色	意味
80%以上	緑（#52c41a）	高い類似度
60%以上	青（#1677ff）	中程度の類似度
40%以上	オレンジ（#faad14）	低い類似度
40%未満	赤（#ff4d4f）	非常に低い類似度

3. プロジェクト名クリック

```
<Tag
  color="green"
  icon={<FolderOpenOutlined />}
  style={{ cursor: onOpenProject ? 'pointer' : 'default' }}
  onClick={() => onOpenProject && hit.project_name && onOpenProject(hit.project_name)}
>
```

```
  {hit.project_name}
</Tag>
```

動作:

- `onOpenProject` が定義されている場合: カーソルをpointerに変更、クリック可能
- クリック時: `onOpenProject(project_name)` を実行

データ型定義

SemanticSearchHit

```
interface SemanticSearchHit {
  transition_set_id: string;          // TransitionSetID（一意識別子）
  transition_id: string;              // TransitionID
  technique?: string;                 // 技術名
  project_name?: string;              // プロジェクト名
  file_path: string;                  // ファイルパス
  score: number;                      // 総合スコア（0-1）
  score_detail: ScoreDetail;          // スコア内訳
  input_text: string;                 // 入力条件テキスト
  transition_text: string;            // 手順テキスト
  output_text: string;                // 出力テキスト
}
```

ScoreDetail

```
interface ScoreDetail {
  input_score: number;                // 入力条件スコア（0-1）
  transition_score: number;           // 手順スコア（0-1）
  output_score: number;               // 出力スコア（0-1）
  structure_score: number;            // 構造スコア（0-1）
}
```

状態別の表示

1. ローディング中

```
if (loading) {
  return (
    <div style={{ textAlign: 'center', padding: 40 }}>
      <Spin size="large" tip="検索中..." />
    </div>
  );
}
```

表示: 中央に大きなスピナー + 「検索中...」

2. エラー発生

```
if (error) {
  return <Alert type="error" message="検索エラー" description={error} showIcon />;
}
```

表示: エラーアラート（赤色、アイコン付き）

3. 未検索

```
if (!searched) {
  return (
    <Empty
      description="検索条件を入力して「検索」ボタンを押してください"
      style={{ padding: 40 }}
    />
  );
}
```

表示: 空状態（検索を促すメッセージ）

4. 検索結果0件

```
if (hits.length === 0) {
  return <Empty description="検索結果が見つかりませんでした" style={{ padding: 40 }} />;
}
```

表示: 空状態（結果なしメッセージ）

5. 検索結果あり

通常のリスト表示

スコア表示

円形プログレス

```
<Progress
  type="circle"
  size={36}
  percent={scorePercent}
  strokeColor={scoreToColor(hit.score)}
  format={(p) => (
    <span style={{ fontSize: 10 }}>{p}%</span>
  )}
/>
```

仕様:

- タイプ: 円形
- サイズ: 36px
- パーセント表示: スコア × 100（整数）
- 色: スコアに応じて動的変更
- フォーマット: 「{パーセント}%」（10px）

スコア内訳タグ

```
<Space size="small" style={{ marginLeft: 8 }}>
  <Tag>入力 {Math.round(hit.score_detail.input_score * 100)}%</Tag>
  <Tag>手法 {Math.round(hit.score_detail.transition_score * 100)}%</Tag>
  <Tag>出力 {Math.round(hit.score_detail.output_score * 100)}%</Tag>
  <Tag>構造 {Math.round(hit.score_detail.structure_score * 100)}%</Tag>
</Space>
```

表示内容:

- 入力: input_score × 100（整数）
- 手法: transition_score × 100（整数）
- 出力: output_score × 100（整数）
- 構造: structure_score × 100（整数）

展開コンテンツ

4軸テキストの折りたたみパネル

```
<Collapse
  size="small"
  ghost
  items={[
    {
      key: 'input',
      label: <Text style={{ fontSize: 11 }}>Input テキスト</Text>,
      children: (
        <Paragraph
          style={{ fontSize: 11, whiteSpace: 'pre-wrap', margin: 0 }}
        >
          {hit.input_text}
        </Paragraph>
      ),
    },
    {
      key: 'transition',
      label: <Text style={{ fontSize: 11 }}>Transition テキスト</Text>,
```

```
      children: (
        <Paragraph
          style={{ fontSize: 11, whiteSpace: 'pre-wrap', margin: 0 }}
        >
          {hit.transition_text}
        </Paragraph>
      ),
    },
    {
      key: 'output',
      label: <Text style={{ fontSize: 11 }}>Output テキスト</Text>,
      children: (
        <Paragraph
          style={{ fontSize: 11, whiteSpace: 'pre-wrap', margin: 0 }}
        >
          {hit.output_text}
        </Paragraph>
      ),
    },
  ]}
/>
```

パネル構成:

1. **Input テキスト**: input_text（改行保持）
2. **Transition テキスト**: transition_text（改行保持）
3. **Output テキスト**: output_text（改行保持）

スタイル:

- サイズ: small
- ゴーストモード（背景色なし）
- フォントサイズ: 11px
- 改行保持: `whiteSpace: 'pre-wrap'`

特記事項

展開状態の管理

Set型を使用:

```
const [expandedIds, setExpandedIds] = useState<Set<string>>(new Set());
```

理由:

- 複数の結果を同時に展開可能
- Set型でIDの存在確認が高速
- 追加・削除が容易

プロジェクト名クリックの動作

条件:

- `onOpenProject` が定義されている
- `hit.project_name` が存在する

動作:

1. プロジェクト名をクリック
2. `onOpenProject(project_name)` を実行
3. 親コンポーネント（TransitionSetSearchModal）で `handleClose()` を実行
4. モーダルを閉じる
5. ProjectManagement画面でプロジェクトを選択

詳細ボタンのアイコン回転

```
<DownOutlined rotate={isExpanded ? 180 : 0} />
```

動作:

- 折りたたみ時: 0度（下向き▼）
- 展開時: 180度（上向き▲）

実装ファイル情報

パス

```
frontend/src/components/TransitionSetSearch/SearchResults.tsx
```

主要な依存パッケージ

- `antd` : UI コンポーネント（List, Typography, Tag, Space, Button, Collapse, Progress, Empty, Spin, Alert）
- `@ant-design/icons` : アイコン（FolderOpenOutlined, FileTextOutlined, DownOutlined）
- `react` : React本体（useState）

変更履歴

バージョン	日付	変更内容	担当者
1.0	2026-03-11	実装ベースで仕様書を新規作成（実装ファイル: SearchResults.tsx）	Cline

---

関連ドキュメント

- TransitionSet検索モーダル
- 重み調整スライダー
- プロジェクト管理画面